



吉林大学

JILIN UNIVERSITY

本科生毕业论文（设计）

中文题目 基于 python 网络爬虫的原神角色数
据分析程序的设计与实现

英文题目 Design and implementation of data analysis
program of Genshin Impact role based on
python web crawler

学生姓名 彭安康

学 号 55190331

学 院 软件学院

专 业 软件工程

指导教师 裴士辉 讲师

2023 年 5 月

基于 python 网络爬虫的原神角色数据分析程序的设计与实现

摘要

本次基于 python 网络爬虫的原神角色数据分析程序选取了原神 Wiki 网站作为爬取对象，并使用了不同种类的爬虫请求、解析、存储技术。使用到了 requests 模块作为爬取工具，re 正则表达式模块、urllib 模块和 lxml 模块进行解析辅助，以及使用 jieba 模块、wordcloud 模块、matplotlib 模块等实现数据的可视化。爬取的内容包括了所有角色名称、角色简介、角色官方介绍、角色故事、角色攻击力属性、防御力属性、生命上限属性、角色图片、角色语音；之后会使用各种存储方式对爬取到的不同种数据进行数据存储以及可视化呈现。其中存储格式包括：txt 文本文档、csv 表格文件、ogg 存储等。可视化包括词云图和柱状图。系统采用了交互式设计，用户可以根据自己需要，手动输入角色名称进行角色信息检索，相应功能模块也可以手动选择。

关键字：

原神角色信息，交互，词云图，爬虫，柱状图

Design and implementation of data analysis program of Genshin

Impact role based on python web crawler

Author: Peng Ankang

Tutor: Pei Shihui

Abstract

The Genshin Impact role data analysis program based on python web crawler selects the Genshin Impact Wiki website as the crawling object, and uses different kinds of crawler request, parsing and storage technologies. requests module is used as the crawl tool, re regular expression module ,urllib module and lxml module for parsing assistance, and jieba module, wordcloud module, matplotlib module for data visualization. The content of crawling includes all the character names, character introduction, character official introduction, character story, character attack power attribute, character defense power attribute, character life cap attribute, character picture, character voice; After that, various storage methods will be used for data storage and visual presentation of different kinds of crawled data. The storage formats include txt text files, csv table files, and ogg storage. Visualizations include word clouds and bar graphs. The system adopts interactive design, users can according to their own needs, manually enter the character name for character information retrieval, the corresponding function module can also be manually selected.

Keywords:

Genshin Impact role information, Interaction, Word Cloud, Crawler, Histogram

目录

第 1 章 绪论	1
1.1 网络爬虫概述	1
1.2 爬虫历史简介	1
1.3 课题现实意义	2
第 2 章 实验设计	3
2.1 爬取对象以及选取理由	3
2.2 开发环境概要介绍	3
2.3 技术涵盖方面概述	4
2.4 实验设计总体思路	4
第 3 章 功能详解	7
3.1 HTTP 请求	7
3.1.1 请求网页	7
3.1.2 角色一览、角色个人信息、角色语音网页	9
3.2 网页解析与数据保存	10
3.2.1 获取所有角色名称	10
3.2.2 获取单个角色简介	13
3.2.3 获取单个角色官方介绍	15
3.2.4 获取单个角色故事	17
3.2.5 获取单个角色属性	20
3.2.6 获取单个角色图片	22
3.2.7 获取单个角色语音	27
3.3 数据可视化	29
3.3.1 角色名称词云	29
3.3.2 角色属性 CSV 表格	33
3.3.3 角色属性柱状图（单个属性 TOP 榜）	37
3.3.4 角色属性柱状图（某一属性某个等级 TOP 榜）	41
第 4 章 结论	46
4.1 系统存在问题	46

4.2 功能完成情况	46
4.3 个人成长心得	47
参考文献	48
致 谢	49

第 1 章 绪论

1.1 网络爬虫概述

网络爬虫，又被称为网页蜘蛛，网络机器人，是一种能够根据一定规律，利用一些规则从目标网页爬取一定数据信息（如，文字，图片，音频，视频等）并保存的自动化程序^[1]。可以代替人们自动地在互联网中进行数据信息的采集与整理。

一个完整的爬虫程序包括三个主要步骤^[2]：

1. 获取网页源代码资源——模拟浏览器向网页发送请求并获取响应，其中响应体即为网页源代码。
2. 筛选出所希望得到的信息——分析网页结构规则根据需求来构造正则表达式或者使用一些封装好的库来处理数据。
3. 存储处理后数据——将爬取的信息采用一定形式（txt 文本，文件夹，csv，数据库等）存储起来方便后续使用。

在爬虫程序中，除了上述三个主要步骤之外，数据可视化也是一个非常重要的步骤。因为在很多场景下我们使用爬虫不单单是为了去获取一些数据，而是更希望用直观意义的方式呈现数据，并且能够快速识别出数据的特征和信息，所以很多情况下数据可视化的步骤也是必不可少的。

1.2 爬虫历史简介

早期爬虫的出现可以追溯到 20 世纪 90 年代初期，当时的互联网还很小，只有数百万个网页。这时候的爬虫主要是为了帮助搜索引擎建立索引，以便更好地为用户提供搜索结果。其中最著名的就是 1993 年诞生的 Lycos 爬虫，它可以从互联网上收集信息并建立索引，成为了当时最流行的搜索引擎之一。

随着互联网的不断发展壮大，爬虫也逐渐成为了各种应用程序的必备工具。例如，在电子商务领域，爬虫被用于抓取商品信息，以便比较价格和评价。在社交媒体领域，爬虫则被用于监测舆情和分析用户行为。

在 21 世纪初期，Google 公司推出了 PageRank 算法，这使得搜索引擎的准确性有了大幅提升。同时，Google 也开发了自己的爬虫 Googlebot，它能够更加智

能地抓取网页，并且能够识别网页中的链接，从而更好地建立索引。Googlebot 成为了当时最流行的爬虫之一，其算法被广泛使用。

在近些年，随着互联网的快速发展，爬虫也迎来了新的挑战 and 机遇。例如，移动互联网的出现使得网页呈现方式发生了很大变化，这对爬虫的抓取和分析提出了新的要求。同时，大数据和人工智能技术的发展，也为爬虫提供了更多的数据源和分析方法，使得其应用领域更加广泛。

1.3 课题现实意义

大数据时代，数据和各个行业领域之间的联系越发紧密，逐渐成为了行业领域不断发展和进步的重要基础。如何从互联网如此海量的数据中获取对自身有利的信息也成了当下大众关注的问题。互联网作为信息数据的沃土，其中生长了大量的数据。若想从如此大规模的数据中寻找有价值的信息，仅凭人力显然是难以解决的。由此，网络爬虫应运而生。网络爬虫可以依照设定的规则对互联网上的数据信息进行自动抓取。爬虫技术可以说是最有效的探寻有价值网络数据信息的渠道。利用 python 语言来发展网络爬虫技术，可以在一定程度上满足网络安全以及产品调研时的数据支持，提升搜索引擎以及数据获取的工作效率^[3]。在当今这个信息就是财富的时代，学会通过爬虫进行信息的高效获取无疑对各行各业每个人发展都有着深远影响。

第 2 章 实验设计

2.1 爬取对象以及选取理由

1. 爬取对象包括:

Bilibili 游戏原神 Wiki 网站 (<https://wiki.biligame.com/ys/>) 中角色相关信息数据, 包括角色名称、角色简介、角色官方介绍、角色故事、角色攻击力属性、防御力属性、生命上限属性、角色图片、角色语音等信息。

2. 对象选取理由:

原神是由上海米哈游网络科技股份有限公司制作发行的一款开放世界冒险游戏, 公测于 2020 年 9 月 28 日开启。游戏发生在一个被称作“提瓦特”的幻想世界, 在这里, 被神选中的人将被授予“神之眼”, 导引元素之力。玩家将扮演一位名为“旅行者”的神秘角色, 在自由的旅行中邂逅性格各异、能力独特的同伴们, 和他们一起击败强敌, 找回失散的亲人——同时, 逐步发掘“原神”的真相。原神用无与伦比的画面把一场场战斗变成了一次次视觉奇观。无论你是在搜索食材、飞越山峰, 还是与其他玩家联手对抗强大 BOSS, 这片大陆的每一寸空间都精雕细琢, 让你感觉不虚此行。毕业设计选择爬取原神 Wiki, 一方面是由于游戏的喜爱, 另一方面也是想将学习和娱乐结合起来, 在娱乐中获取更多知识。

2.2 开发环境概要介绍

基于 python 网络爬虫的原神角色数据分析程序是在 windows10 环境下开发的, 采用 python 3.9 版本作为开发语言, 并结合以 JetBrains PyCharm Community Edition 2019.1 x64 的 IDE 开发环境来进行代码编写与优化; 此外, 由于爬虫需要通过浏览器向网页发送请求和获取网页信息, 模拟了多个版本的火狐等浏览器, 并且在模拟浏览器之后, 还需要进行发出请求, 解析响应, 保存数据, 调用数据等一系列操作, 此时使用了许多 python 模块, 如发出请求的 requests 模块, 解析网页结构的 lxml 模块、正则表达式 re 模块等。

2.3 技术涵盖方面概述

Python 是一种面向对象的高级通用脚本编程语言，其语法简洁、简单易懂^[4]。Python 语言因其自身简单和丰富资源库的特性，在网络爬虫和数据分析方面有着巨大的优势^[5]：

1. HTTP 请求
 - (1) requests 库——第三方请求库
2. 页面解析
 - (1) lxml——HTML 或 XML 解析库
 - (2) 正则表达式——万能解析工具
 - (3) urllib.parse——解析模块，用于解析 URL
3. 数据存储
 - (1) CSV 文本存储——EXCEL 表格呈现数据
 - (2) TXT 文本存储
 - (3) 文件夹存储
 - (4) 音频文件存储——“.ogg”格式存储
4. 数据可视化
 - (1) Word Cloud（词云图）+jieba 分词统计
 - (2) 柱状图——matplotlib（一个 Python 的 2D 绘图库）

2.4 实验设计总体思路

工欲善其事，必先利其器。要应用爬虫技术，首先要看应用的领域，本文主要针对网页的结构，了解网页的结构，这些光鲜亮丽的页面都是由代码组成的。这种代码我们称之为 HTML（超文本标记语言），HTML 是浏览器（Chrome、Safari、IE、Firefox 等）看得懂的语言^[6]。我们在正式地开始编写代码之前需要对想爬取的网站结构和内容有一定的分析和理解，如图 2-1 所示，原神 Wiki 有角色图鉴、武器图鉴等内容，由于本程序为原神角色数据分析程序，所以我们重点关注角色图鉴这一栏。



图 2-1 原神 Wiki 排版

打开角色图鉴后，我们能看到角色一览页面，如图 2-2，通过此页面我们可以获取所有角色的名称以及角色详细个人信息的网址，极大地便利了后续对所有角色信息的爬取流程。

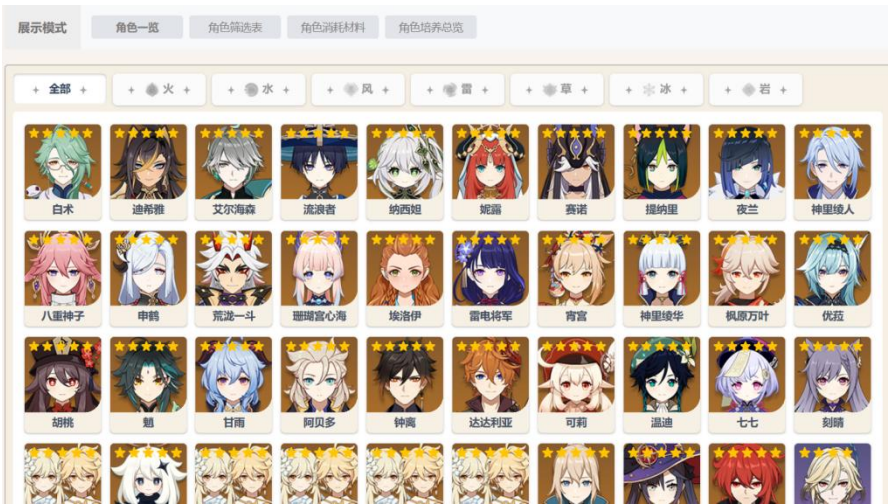


图 2-2 原神角色一览

随机点开一个角色后，通过目录可以发现其中每个角色的界面包含的信息大致如下图 2-3 所示：



图 2-3 角色信息目录

从上述目录可以得知每个角色的相关信息种类繁多且复杂，所以本设计最终决定挑选其中的角色名称、角色简介、角色官方介绍、角色故事、角色攻击力属性、防御力属性、生命上限属性、角色图片、角色语音数据信息进行爬取。在明确爬取目标之后，我们需要考虑对应内容的保存方式。在最后采用到的数据结构类型有数组以及字典来进行保存，少量内容以字典数组的方式保存。决定好数据结构后，下一步便是将爬取后处理好的数据保存到计算机本地以防止得到的数据信息丢失，实现持久化存储。本程序使用到的本地存储形式有 csv 格式、txt 格式和 ogg 格式。其中 txt 格式使用广泛，操作简单。csv 文件则可以以 excel 表格格式打开，并且还有 excel 文档小巧简单，方便且灵活，几乎每个人都看得懂，并且能够使用一些基本的操作的优点，而且如今几乎每台电脑都有配备 EXCEL 表格，泛用性比较强。不过总体而言上述存储形式还是不如使用数据库存储专业，也不适合用于正式开发场景，仅适用于私人的，小型的项目。

在爬取、处理并保存数据之后，还会使用一些可视化相关的技术来实现云图、柱状图等可视化功能以便更直观的观测数据。其中使用到的模块有 jieba 模块、wordcloud 模块、matplotlib 模块。

第 3 章 功能详解

3.1 HTTP 请求

HTTP 请求既是网络爬虫的第一步，又是最为重要的一步，因为后续数据处理的来源便是发送请求后获取的网页响应。如果因为某些原因导致网页请求失败，那么接下来的全部工作都将无法继续进行。

3.1.1 请求网页

1. 程序源码片段详解

发送 HTTP 请求是爬虫程序的第一步，也是非常关键的一步，在 `get_requests` 函数中，我们需要传入目标网站的 `url`、模仿浏览器的 `headers` 以及 `ip` 代理地址 `proxies` 作为函数参数，传递 `headers` 是由于在默认的情况下，爬虫发送请求时会在请求头里直接标明自己是爬虫程序，若是遇到拒绝爬虫访问的网站会被直接拒绝访问，所以在访问时需要使用 `headers` 进行伪装，例如在 `headers` 中添加 `User-Agent`，`headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36"}`

而 `get_requests` 方法具体实现如下：

```
def get_requests(url, headers, proxies):  
    try:  
        res1 = requests.get(url, headers=headers, proxies=proxies) # 获取网页内容  
    except:  
        print("connect error")  
        res1 = get_requests(url, headers, proxies)  
    return res1
```

此处首先使用到了 `requests` 模块中的 `get` 函数，`requests.get()` 方法至少需要一个参数。（接口的地址），其为有返回值的方法。返回值就是本次请求的服务端响应结果。`headers` 和 `proxies` 则在上文已说明。可以观察到方法中在调用

requests 的 get 方法时使用了 try-except 语句,并在 except 捕获到异常后再次进行了一个 get_requests 方法的递归调用,这是因为如果在访问目标网站时由于某些不确定因素导致目标访问失败则可能导致后续功能无法执行甚至程序崩溃,所以该方法采用了递归调用的策略,直至访问成功后再返回目标网页的响应体 res1。

而参数列表中的 headers 和 proxies 使用了下述方法随机抽取,因为某些网站会限制同一 ip 地址短时间内的访问次数,为了提高爬虫的爬取效率本程序从 <https://www.kuaidaili.com/free/inha> 这一国内免费代理 ip 网页中获取了数个高匿代理 ip 存储在本地 IP.txt 文件中,再在主程序中进行读取并存在 item 这一字典对象数组中,并在使用时通过 random 模块的 choice 方法随机选取一个高匿 ip 地址进行访问,代码如下所示:

```
f = open("IP.txt", "r")

file = f.readlines()

# 遍历并分别存入列表,方便随机选取 IP

item = []

for p in file:

    p = eval(p.replace("\n", ")) # 以换行符分割,转换为 dict 对象

    item.append(p)

f.close()

proxies = random.choice(item) # 随机选取一个 IP
```

同理,headers 也是采用了 random 模块的 choice 方法随机选取:

```
headers['User-Agent'] = random.choice(user_agent_list)

res1= get_requests(url, headers, proxies)
```

之后我们便可以通过 get_requests()方法获取所需的网页,并用 response 来接收网站返回的响应体。由于可能会有编码方式的不同,所以我们采用了 decode() 对读取内容进行解码。

```
res2 = res1.text.encode('gbk', 'ignore').decode('gbk')
```

此时, res2 变量中就保存了我们需要的网页源码,我们需要将其返回给调用函数,以待后续加工处理。

3.1.2 角色一览、角色个人信息、角色语音网页

截止至 2023 年 5 月 17 日星期三，原神总共推出了 67 名不同的角色，每位角色的信息都包含在各个角色自己的详细信息页面里，为了获得所有角色信息本程序至少需要发送网页请求 67 次，以此来得到这所有 67 名角色各自的信息页面的响应体。而这 67 个角色的个人信息网页链接都有着相同的前缀

`https://wiki.biligame.com/ys`，唯一不同之处就在于网页的后缀为各个角色的名称，例如，只需要修改网页后缀为“刻晴”，即“`https://wiki.biligame.com/ys/刻晴`”，我们便可以爬取到刻晴这一角色的详细信息页面，所以每次爬取一个角色，只需要提供网页前缀和后缀即可。

1. 获取角色一览网页

由于角色一览的网址就是“`https://wiki.biligame.com/ys/角色`”没有变化，所以我们可以直接访问该网址获取网页内容，代码如下所示：

```
origin_res = requests.get("https://wiki.biligame.com/ys/角色", headers=headers) # 获取网页内容

origin_res_text = origin_res.text.encode('gbk', 'ignore').decode('gbk') # 使网页文本格式为 gbk

origin_res.close()
```

2. 获取单个角色个人信息网页

在获取单个干员网页的方法中，我们以 `base_url` 和 `character_name` 作为函数的参数，前者是网页的前缀，后者则是角色的名称。

```
origin_url = "https://wiki.biligame.com/ys/"

proxies = random.choice(item) # 随机选取一个 IP

headers['User-Agent'] = random.choice(user_agent_list)

res1 = get_requests( origin_url + name, headers, proxies) # 捕捉异常解决连接断开
```

3. 获取角色语音页面

由于角色语音并不在角色个人信息页面中，而是在一单独的角色语音页面。通过观察可以发现角色语音的网址格式为“`https://wiki.biligame.com/ys/`”+角色名称+“语音”，所以我们可以模仿获取单个角色个人信息网页的方式去访问角

色语音网页，如下所示：

```
res1 = requests.get("https://wiki.biligame.com/ys/" + name+"语音")
if str(res1) == '<Response [404]>': # 请求页面不存在
    print('请求 404')
exit()
```

3.2 网页解析与数据保存

3.2.1 获取所有角色名称

1. 综述

通过前面 3.1.2 所述，当程序想访问每个角色个人信息的网页和角色语音网页时需要两部分，一部分是固定的公共网络地址前缀，而另一部分则分别是角色名称以及角色名称+“语音”。所以为了后续爬取分析角色信息的功能得以实现，本程序需要先获取所有的角色名称。而通过 2.4 的实验设计总体思路可以发现，在角色一览这一页面中就存有本程序所需的所有角色的名称。因而先进行角色一览这一页面的爬取和解析。

2. 程序片段详解

在经过 3.1.1 HTTP 请求获取网页后就可以开始进行网页内容分析从而获取想要的数内容。通过分析网页的源码得知，角色的名称以及角色详细页面的超链接都包含在第一个 resp-tab-content 类的 div 块中，每个角色名称和超链接又分别包含在<div class="L">标签和一个<a>标签内，最后有两种方法获取角色名称，一个是获取 div 标签内的内容，另一个是直接切割<a>标签内的超链接。并且就如何提取出上述两者也有多种方法。本程序使用了两种方法进行信息提取，第一种是正则表达式，这种方法较为原始，需要根据上下文特征一步步截取出文字块。第二种则使用了 lxml 模块进行解析，此模块可根据给出的标签类型和标签内属性进行自动筛选，此种方法可极大提高分析效率。以下是两种方法获取角色名称的程序步骤：

2.1 re 正则表达式模块：

(1) 先过滤出 resp-tab-content 类的 div 块：

```
pattern1 = re.compile(u'resp-tab-content.*resp-tab-content', re.S)
res1 = pattern1.search(res).group(0)
```

此处 `re.compile()` 方法为 `re` 正则表达式模块下的一个方法，该方法可以将正则表达式的字符串形式编译为一个 `Pattern` 对象。其中需要传两个参数，第一个参数为正则表达式格式的字符串，用于寻找匹配该格式的字符串；第二个参数为 `flag`，`flag` 是一个可选参数，表示匹配模式，比如忽略大小写，多行模式等。在本程序中将会使用到 `re.S` 这一匹配模式，表示忽略换行符。

获取正则表达式的 `pattern` 对象之后就可以像第二行一样使用其内置的 `search` 方法查找父字符串内符合该正则表达式格式的第一个子字符串。

(2) 获取包含每个角色信息超链接的 `<div class="L">` 标签块：

```
pattern2 = re.compile(u'<div class="L">.*?title', re.S)
res2 = pattern2.findall(res1)
```

此处仍旧是使用 `re.compile` 方法创建出一个 `pattern` 对象，但下一步没有使用 `search` 方法，而是其内置的另一个匹配方法 `findall`。`search` 和 `findall` 方法有一定的区别，具体表现为，`re.search` 函数可以在字符串中查找第一个匹配正则表达式的子串并返回一个匹配对象；`re.findall` 函数可以在字符串中查找所有匹配正则表达式的子串，并返回一个列表。总体而言 `search` 只查找第一个匹配的字符串，而 `findall` 会遍历查找整个字符串里所有子字符串。

(3) 将角色名称从超链接中切割出来并存到文件中：

```
pattern3 = re.compile(u'/ys.*? "', re.S)
file2 = open('character_name.txt', mode='w', encoding='utf-8')
file3 = open('character_url.txt', mode='w', encoding='utf-8')
```

两小段代码分别打开了角色名称文件和角色个人信息网页地址后缀文件，以便后续实现将角色名称数据存入各自 `txt` 文档的功能，并设置编码格式为 `'utf-8'`（用来使文件支持中文的字符），对文件操作方式为 `'w'`。

【注】“w”：以重写方式打开文件。倘若欲存储的文件名已经存在，则将其覆盖重写，若该文件在目标目录中不存在，则在目标目录下创建新的文件。

```
for link in res2:
    link = pattern3.search(link).group(0)
```



```
link = link.rstrip("\\")

link = link.replace('/ys/', '')

decoded_url = urllib.parse.unquote(link)

file3.write(link + '\n')

file2.write(decoded_url+'\n')

print(decoded_url)

all_url.append(link)
```

此段 for 循环中主要对所获取到的超链接进行分割处理以及将编码为 url 格式下的中文字符重新解码为中文。

分割超链接主要使用到了 python 中字符串自带的 replace 方法。Python 中选中字符串中的一段字符替换成新的字符的方法，通常带两个参数，第一个是旧字符，第二个是新字符。如果指定第三个参数 max，则替换不超过 max 次。

将编码后的字符解码为中文字符这一功能则使用到了 urllib.parse 这一模块。通过调用 urllib.parse.unquote 函数，将其传入编码后的字符串作为参数，该函数会返回解码后的中文字符。

该方法运行结果截图如下：

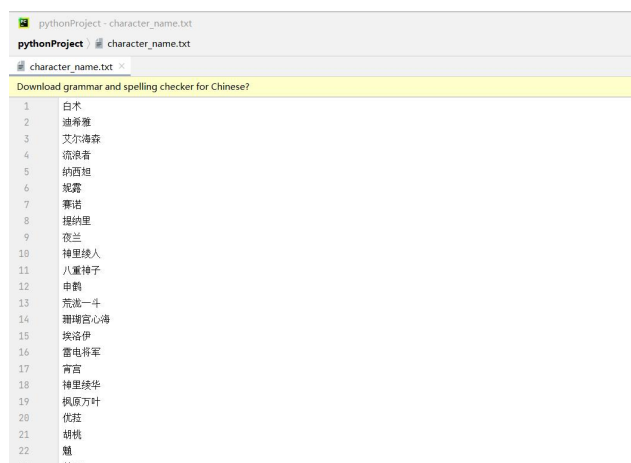


图 3-1 正则表达式方法获取角色名称截图

2.2 lxml 模块：

使用 lxml 模块的情况下根据上述网页结构分析可通过下列语句实现功能。

(1) 先将获取到的网页内容通过 etree.HTML 进行转化成 lxml 格式对象：

```
html_data = etree.HTML(origin_res_text)
```

此段代码通过调用 HTML 类对 HTML 文本进行初始化，成功构造 XPath 解析对象，同时可以自动修正 HTML 文本（标签缺少闭合会自动添加上）

（2）使用其 xpath 方法获取到程序所需所有角色名称：

```
names=html_data.xpath( '//div[@class="resp-tabs-container"]/div[@class="resp-tab-content"]/div[@class="resp-tab-case"]/div/div[@class="L"]/text()')
```

此段代码通过 xpath 筛选出所需要的代码信息，XPath（XML Path Language）是一门在 HTML/XML 文档中查找信息的语言，可用来在 HTML/XML 文档中对元素和属性进行遍历。在 Python 爬虫中，我们可以利用 XPath 快速地定位 HTML/XML 响应中的特定元素以及获取节点的信息，并且通常情况下会比使用正则表达式提取更简单而且更高效。

（3）将获取到的全部角色名称存入本地文件：

```
for name in names:
    file.write(name+'\n')
    print(name)
```

该方法运行结果同正则表达式方法结果，就不再重复截图展示。

3.2.2 获取单个角色简介

1. 综述

实现该功能所需的两个参数为公共网页地址前缀 `original_url` 和角色名称 `character_name`，有了这两个参数之后程序就可以访问单个角色的个人详细信息页面了，之后再根据其网页结构进行分析处理就能得到所需的单个角色的简介。首先是网页结构的分析，通过观察可发现角色个人简介的数据保存在一个 `<table class="wikitable" style="margin-top:0px;width:100%">` 下的 `<tbody>` 标签中，但是由于该网页中存在多个此种结构，如若使用 `lxml` 模块可能会存在分析繁琐以及出现顺序错误的情况，所以此处本程序选择使用 `re` 正则表达式进行处理。再次观察可发现该 `table` 标签前必定存在一个临近的“称号”二字，所以依此可定位出存放角色简介的 `table` 位置并获取该数据块。后续再处理掉数据块无用的 `html` 标签便可得到角色简介数据。

2. 程序片段详解

(1) 获取单个角色的个人信息页面:

```
res1 = requests.get(original_url + name, headers=headers)

if str(res1) == '<Response [404]>': # 请求页面不存在
    print('请求 404')
    exit()

res2 = res1.text.encode('gbk', 'ignore').decode('gbk')
```

此处首先对网页进行请求并将响应结果存入 `res1` 中, 后续有一个 `if` 语句用来判断是否成功读取页面, 如果失败则给出提示并关闭程序, 如果成功则将响应体转化为 `gbk` 格式存入 `res2` 中。

(2) 筛选出存放角色个人简介信息的 `table` 数据块:

```
pattern1 = re.compile("<tr><th>称号.*?</tbody>", re.S)

str = pattern1.search(text).group()
```

此处代码块前加上 `<tr><th>` 是为了方便后续数据处理。

(3) 根据规律将代码块中 `<tr>` 和 `<th>` 分别转化为 `'\r'` 和 `'.'`, 再将特殊字符 `' '` 从数据块中去掉, 并将字符串 `'5 星'` 和 `'4 星'` 分别转化为 `'>5 星<'` 和 `'>4 星<'` 以便后续数据处理。

```
str = str.replace('</th>', '.') # 键值对中键和值的分割

str = str.replace('</tr>', '\r') # 每组键值对之间的分割

str = str.replace('&#160;', '')

str = str.replace('5 星', '>5 星<')

str = str.replace('4 星', '>4 星<')
```

(4) 找出所有的 `html` 标签并将其从数据块中全部去除, 并将数据存入本地文件中:

```
slist = pattern2.findall(str) # 找出 html 标签

for s in slist: # 去除 html 标签
    print(str)

    str = str.replace(s, "")

str = str.replace('\n', "")

file1.write(str)
```

图示 3-2 为此功能运行结果截图

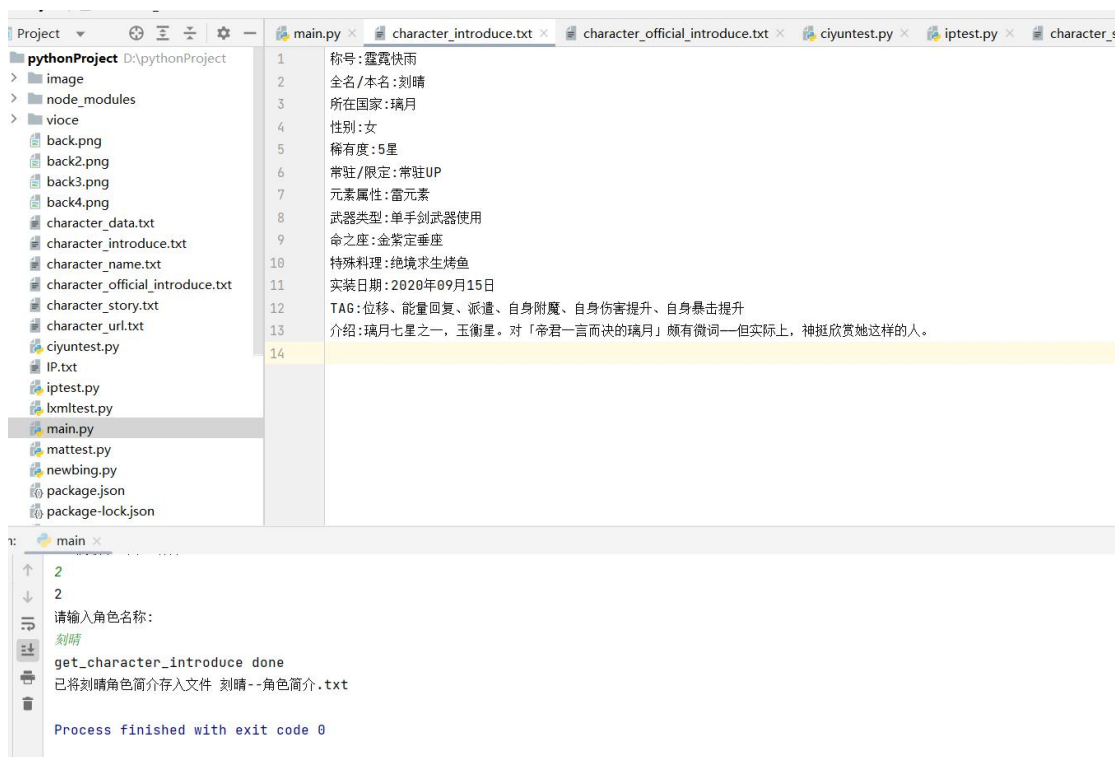


图 3-2 获取单个角色简介

3.2.3 获取单个角色官方介绍

1. 综述

本功能及后续除获取角色语音功能外都在角色个人信息页面进行分析处理，此处就不再重复上面 3.2.2 的页面爬取过程，直接开始页面分析部分。经观察发现，角色官方介绍这一数据存放在一个<div class="mw-parser-output">标签下的<p>标签中，并且该<p>标签没有任何属性特征以及在其父 div 块中有众多类似的无任何属性表示的<p>标签，由此观察 lxml 模块并不太适用于此处。所以该功能的分析工具确定为 re 正则表达式模块。

再经观察发现，角色官方介绍内容必定包含在了“官方介绍”四字与“剧情相关”四字之间，由此可先提取出这一数据块用于缩小范围。但是，此时的数据块中仍旧保有一部分上述提及的干扰的<p>标签，所以下一步仍需继续缩小数据块。第二步分析发现此时可再在缩小后的数据块基础上由“壹·人物”这一必然出现的标题为首即可去除其余无用的<p>标签。经此操作后便可得到角色官方数据的雏形，后续仅需去除掉一些无用的 html 标签即可。

2.程序片段详解

(1) 筛出包含角色官方介绍信息的数据块，以“官方介绍”四字为首，以“剧情相关”四字为尾：

```
pattern1 = re.compile("官方介绍.*?剧情相关", re.S)
res1 = pattern1.findall(text)[1]
```

(2) 进一步缩小数据块以排除其他无用<p>标签的干扰：

```
pattern2 = re.compile("壹·人物.*", re.S)
res2 = pattern2.search(res1).group()
```

(3) 去除掉无用的 html 标签以及'\n'等特殊字符：

先给出 re 正则表达式

```
pattern3 = re.compile("<.*?>", re.S)
pattern4 = re.compile("<.*?剧情相关", re.S)
```

```
slist = pattern3.findall(res2) # 找出 html 标签
```

此处使用正则表达式中的 findall 函数找出所有的 html 标签，以便后续去除。

```
for s in slist: # 去除 html 标签
```

```
    res2 = res2.replace(s, "")
```

该 for 循环会遍历上行代码中获取的所有 html 标签列表，之后再通过字符串自带的 replace 函数将其替换为空，从而达到实现去除 html 标签的目的。

```
s = pattern4.findall(res2)[0]
res2 = res2.replace(s, "")
res2 = res2.replace('\n', "")
```

这几段代码则将前面用于定位的无用信息以及换行符给去除掉

```
file1 = open("character_official_introduce.txt", "w")
file1.write(res2)
file1.close()
```

此段代码则是进行文件操作，将处理好的信息写入文件中并关闭文件以保证文件安全。

程序运行结果如图 3-3 所示：

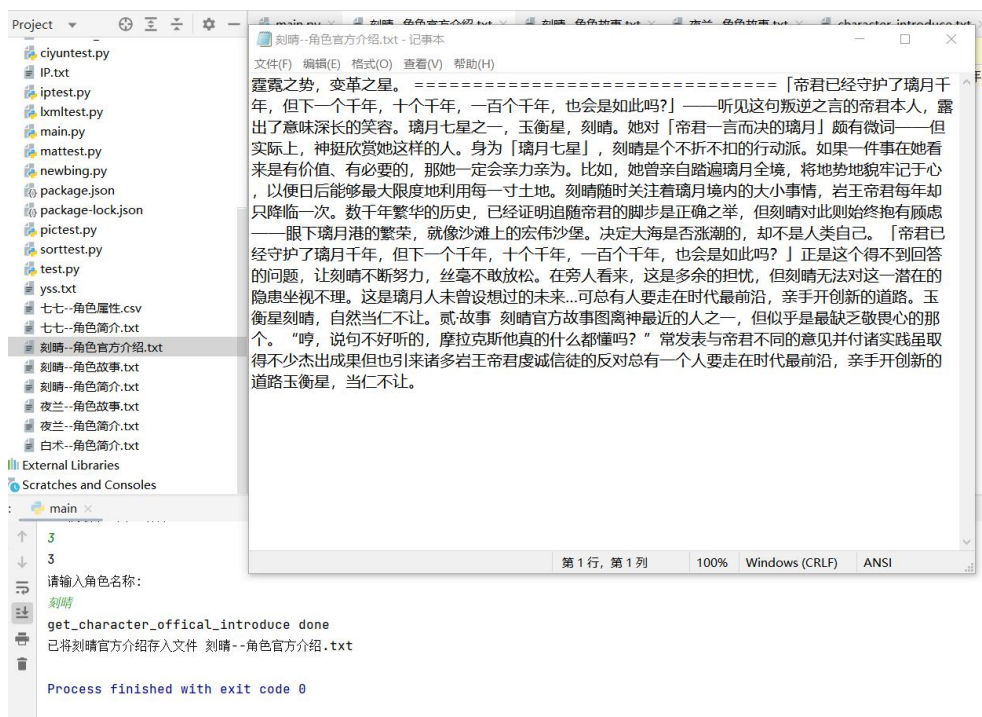


图 3-3 获取单个角色官方介绍

3.2.4 获取单个角色故事

1. 综述

本功能网页访问部分同 3.2.2。此处直接开始页面分析部分。经观察发现，角色故事这一数据存放在一个<div class="poke-bg" style="color: #16769c">标签下的<div class="poke-bg">标签中，并且在其父 div 块中有众多其他类似的<div class="poke-bg">标签，由此观察 lxml 模块并不太适用于此处。所以该功能的分析工具确定为 re 正则表达式模块。

继续观察发现，角色官方介绍内容必定包含在了“角色详细”四字与标签</td>之间，由此可先提取出这一数据块用于缩小范围。由于每个角色都存在数个故事，包括角色详细、角色个人故事以及角色神之眼故事，而每个故事内容又分别存在各自<tr>标签下的<td>标签中，所以此时需要对数据块进行进一步处理。

通过观察可发现每个角色的各个故事标签都存在 story+数字的标记，所以可以根据这一特征进行筛选处理。

2.程序片段详解

(1) 筛出包含角色故事的数据块, 以“角色详细”四字为首, 以标签</td>为尾:

```
pattern0 = re.compile("角色详细.*?</td>", re.S)
res2 = pattern0.search(text).group()
```

(2) 根据每个角色的各个故事标签都存在 story+数字的标记处理, 提取出该角色的所有故事:

```
for i in range(1, 10):
    pattern1 = re.compile("story" + i.__str__() + ".*?</td>", re.S)
    str = pattern1.search(text)
```

上述两行代码先简要的提取出了单个故事的数据块。

```
pattern2 = re.compile("story.*?>", re.S)
if str:
    res1 = str.group()
    story_lable = pattern2.search(res1)
    res1 = res1.replace(story_lable.group(), "") # 去除 story 标签
    res2 = res2 + res1
```

这段代码则是先对 str 进行判断, 如果 str 不为空也就是找到了某一故事, 则通过 pattern2 找出其用于定位的 story+数字标签, 再通过 replace 函数将其去除掉。

通过观察发现每个角色故事都不超过 8 个, 但是保险起见, 该 for 循环从 1 开始遍历到 10。

(3) 去除掉无用的 html 标签:

```
pattern3 = re.compile("<.*?>", re.S)
slist = pattern3.findall(text) # 找出 html 标签
```

此处使用正则表达式中的 findall 函数找出所有的 html 标签, 以便后续去除。

```
for s in slist: # 去除 html 标签
    res2 = res2.replace(s, "")
```

该 for 循环会遍历上行代码中获取的所有 html 标签列表，之后再通过字符串自带的 replace 函数将其替换为空，从而达到实现去除 html 标签的目的。

```
s = pattern4.findall(res2)[0]
```

```
res2 = res2.replace(s, "")
```

```
res2 = res2.replace("\n", "")
```

这几段代码则将前面用于定位的无用信息以及换行符给去除掉

```
file1 = open("character_story.txt", "w")
```

```
file1.write(res2)
```

```
file1.close()
```

此段代码则是进行文件操作，将处理好的信息写入文件中并关闭文件以保证文件安全。

程序运行结果如图 3-4 所示：

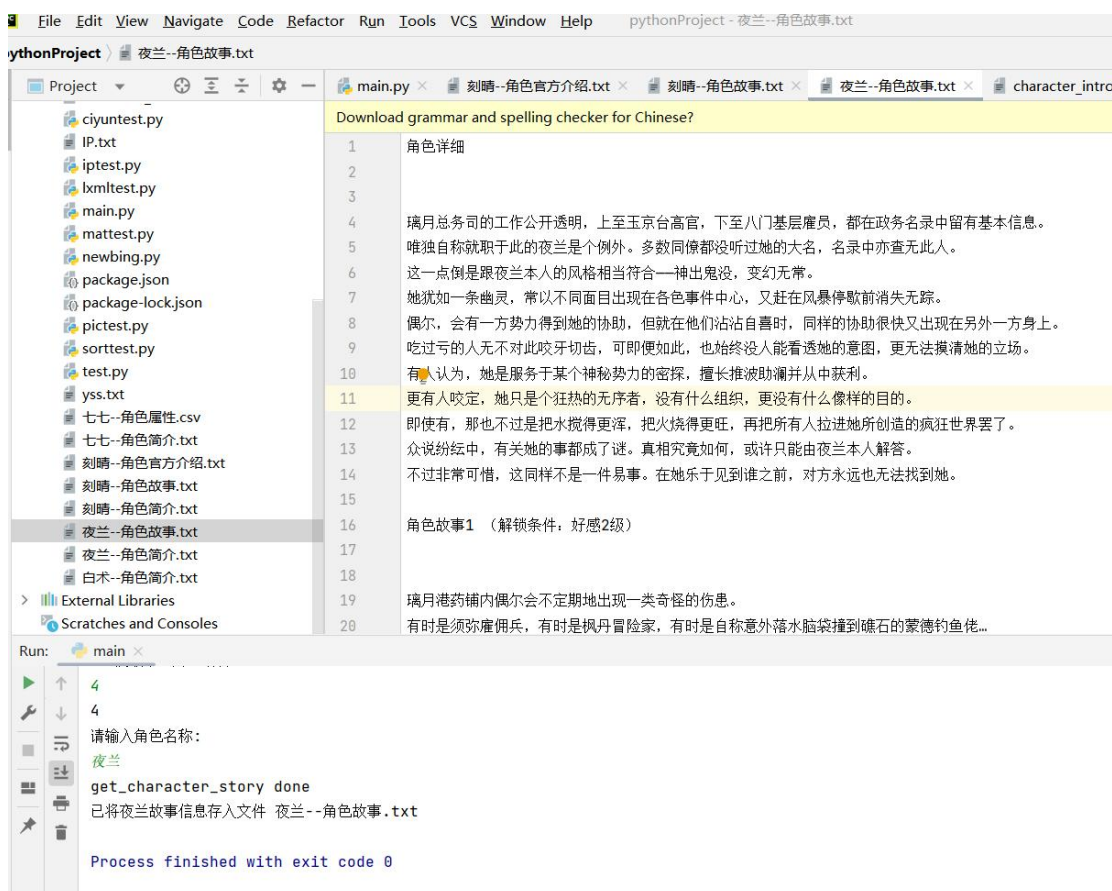


图 3-4 获取单个角色故事

3.2.5 获取单个角色属性

1. 综述

本功能网页访问部分同 3.2.2。此处直接开始页面分析部分。经观察发现，角色数据这一信息同 3.2.4 一样存放在同一个 div 块下的<div class="poke-bg">标签中，并且在其父 div 块中有众多其他类似的<div class="poke-bg">标签，由此观察 lxml 模块并不太适用于此处。所以该功能的分析模块选择为 re 正则表达式模块。

继续观察发现，角色官方介绍内容必定包含在了“”这一标签</tbody>之间，由此可先提取出这一数据块用于缩小范围。此后可再单独提取出<tbody>与</tbody>之间的内容即可初步获取到角色属性数据。

2. 程序片段详解

(1) 筛出包含角色故事的数据块，以“id="属性数据"”为首，以标签</tbody>为尾：

```
pattern1 = re.compile("id=\"属性数据.*?</tbody>", re.S)

if pattern1.search(text) is None:

    return None
```

此段的 if 语句用于判断该角色是否有属性这一数据，因为在原神 Wiki 网站中有派蒙这一角色。该角色为游戏中的向导角色，仅具有游戏指引以及引导剧情的作用，玩家并不能对其进行操作，其也没有攻击力、防御力和生命上限这些基础属性，所以当爬取过程中访问到该网页时会存在无法使用正则表达式筛出数据块的情况，此时 search 函数会返回一个 None 值。如若 search 函数返回 None，该功能函数也返回 None。

(2) 对<tbody>标签内数据进行处理：

```
res1 = pattern1.search(text).group(0)

res2 = pattern2.search(res1).group(0)

res2 = res2.replace('</th>', ',')

res2 = res2.replace('</td>', ',')
```

```
res2 = res2.replace('</tr>', ';')
res2 = res2.replace(',', ';')
res2 = res2.replace('\n', '')
res2 = res2.replace('\r', '')
res2 = res2.replace('\t', '')
```

此段则是对<tbody>标签中数据进行处理以便后续可视化功能的实现，具体有以下操作：

- A. 将'</th>'、'</td>'替换为 ';'字符。
- B. 将'</tr>'、';'替换为';'字符。
- C. '\n'、'\r'、'\t'等特殊字符去除。

(3) 去除掉剩余的 html 标签：

```
pattern3 = re.compile("<.*?>", re.S)
slist = pattern3.findall(res2) # 找出 html 标签
for s in slist: # 去除 html 标签
    res2 = res2.replace(s, '')
```

(4) 将处理好的数据写入本地文件中：

```
file1 = open("character_data.txt", "w")
file1.write(res2)
file1.close()
```

此段代码则是进行文件操作，将处理好的信息写入文件中并关闭文件以保证文件安全。

程序运行结果如图 3-5 所示：

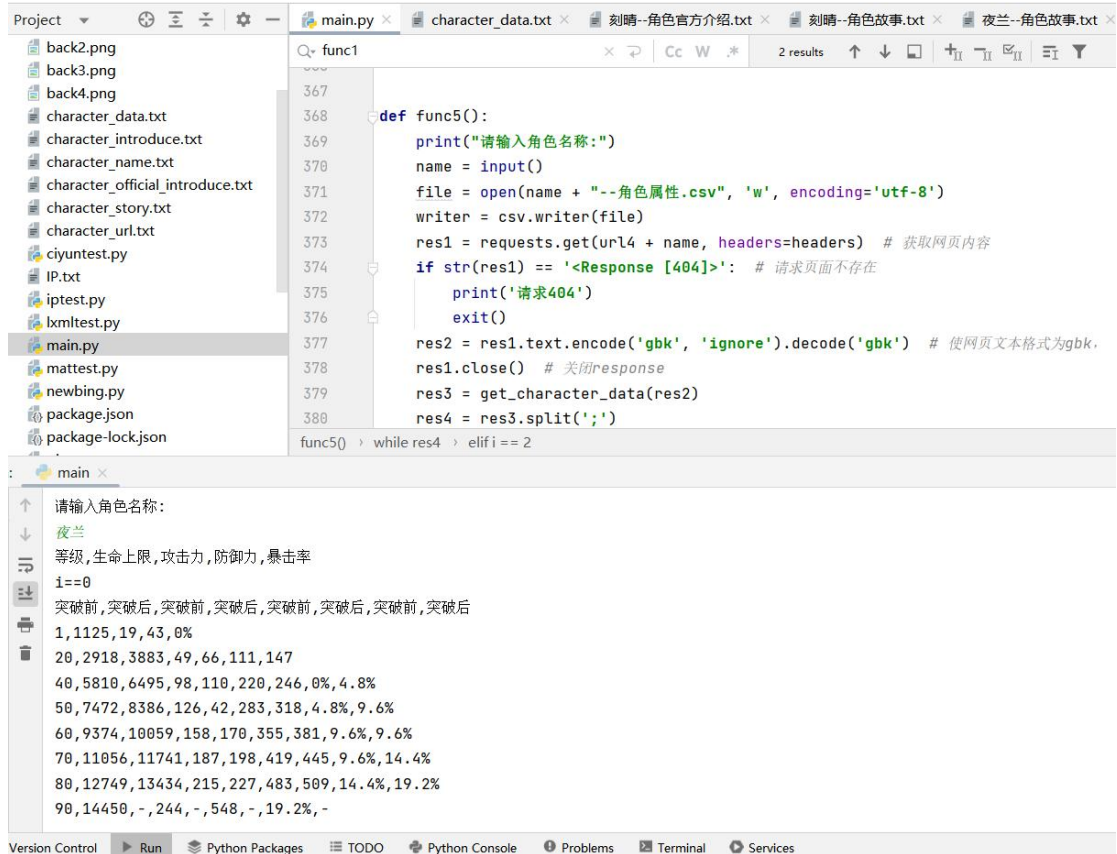


图 3-5 获取单个角色数据

3.2.6 获取单个角色图片

1. 综述

本功能网页访问部分同 3.2.2。此处直接开始页面分析部分。经初步观察发现，角色图片在原神 Wiki 的角色个人信息网页中总共可分为立绘、角色官方介绍图和角色官方贺图三部分，所以本功能也可分为三个小功能进行页面分析。该功能的分析模块选择为 re 正则表达式模块。

首先是立绘部分。通过观察可发现立绘图片存在于 `<div class="col-sm-4">` 标签下的 `<div class="resp-tabs-container">` 中，其中每个立绘又分别存在其各自 `<div class="resp-tab-content">` 标签下的一个 `<a>` 超链接标签中，但由于该标签下仍有其他 div 干扰项，此处仍旧选择了 re 正则表达式进行解析，防止解析出现错误。再次观察发现该部分必定包含在以 “id=“立绘”” 这一字符串以及 “基本信息” 四字之间。所以此时需要根据上述特征获取到该 `<a>` 标签内部的图片地址。

其次是角色官方介绍图部分。初步观察可发现该图片所处位置在角色官方介绍处，处于“官方介绍”四字以及“官方介绍图”五字之间，并且该部分数据块中没有其他地址干扰项，可以直接通过 `http` 作为标志获取图片地址。

最后是角色官方贺图部分。在角色个人信息页面中，角色官方贺图是一个相对比较简洁的部分。该部分中几乎仅存有一些文字、图标以及官方贺图，所以可以较为方便地获取到官方贺图的图片地址。

2. 程序片段详解

2.1 立绘部分

(1) 筛出包含角色立绘的数据块，以“`id=\"立绘\"`”为首，以“基本信息”四字结尾：

```
pattern1 = re.compile("id=\"立绘\".*?其它信息", re.S)#筛出立绘部分
lihui_part = pattern1.search(text).group()
```

(2) 创建该角色在 `image` 路径下的专属文件夹：

```
if not os.path.exists("image/" + name):
    print("文件不存在")
    os.mkdir("image/" + name)
```

此处使用到了 `os` 模块，`os` 即 `operating system`（操作系统），`Python` 的 `os` 模块封装了常见的文件和目录操作。

`os.path` 模块主要用于文件的属性获取，`exists` 是“存在”的意思，所以顾名思义，`os.path.exists()` 就是判断括号里的文件是否存在的意思，括号内的可以是文件路径。

`os.mkdir(path)` 函数的功能创建目录。`os` 模块中存在另一与其相似的函数 `os.makedirs(path)`。两者存在一定区别。

`os.mkdir(path)` 的功能是一级一级的创建目录，前提是前面的目录已存在，如果不存在会报异常，虽然比较麻烦，但是当目录是根据文件名动态创建的时候会发现他虽然繁琐但是很有保障，不会因为某些未知错误，而创建了双层或者多层错误路径。

`os.makedirs(path)` 可以一次创建多级目录，哪怕中间目录不存在也能正常的替

你创建而不进行报错，存在一定风险，所以此处选择调用 `os.mkdir(path)` 函数。

该 if 语句通过 `os.path.exists()` 函数的返回值来判断该角色文件夹是否已存在。如果不存在则通过 `os.mkdir(path)` 函数创建。

(3) 获取所有立绘的图片地址并存入 `image` 文件夹下该角色名称的文件夹中：

```
pictrues = pattern3.findall(lihui_part)
i = 1
```

此处 `i` 用来区分各个立绘。

```
for p in pictrues:
    pic = p.replace("\\", "")
    print(pic)
    r = requests.get(pic)
    f = open("image/" + name + "/" + "立绘" + i.__str__() + "." + pic.split('.').pop(), "wb")
```

此处由于写入内容为图片，所以文件应以二进制格式打开，属性为“wb”。

```
f.write(r.content)
f.close()
i = i + 1
```

2.2 角色官方介绍图部分

(1) 筛出包含角色官方介绍图的数据块，以“id=“官方介绍””为首，以“官方介绍图” 五字结尾：

```
pattern1 = re.compile("id=\"官方介绍\".*?官方介绍图", re.S)
gip_part = pattern1.search(text).group()
lihui_part = pattern1.search(text).group()
```

(2) 创建该角色在 `image` 路径下的专属文件夹：

```
if not os.path.exists("image/" + name):
    print("文件不存在")
    os.mkdir("image/" + name)
```

此处 `os` 模块用法同上立绘部分。

(3) 获取所有立绘的图片地址并存入 `image` 文件夹下该角色名称的文件夹中:

```
pictrue = pattern2.search(gip_part).group().replace("\\", "")
r = requests.get(pictrue)

f = open("image/" + name + "/" + "官方介绍图" + "." + pictrue.split('.').pop(), "wb")
f.write(r.content)
print(r.content)
f.close()
```

2.3 角色官方贺图部分

(1) 筛出包含角色官方贺图的数据块, 以 “`id="官方贺图"`” 为首, 以 “角色宣传视频” 六字结尾:

```
pattern1 = re.compile("id=\"官方贺图\".*?角色宣传视频", re.S) # 筛出官方介绍图部分
con_part = pattern1.search(text).group()
```

(2) 创建该角色在 `image` 路径下的专属文件夹:

```
if not os.path.exists("image/" + name):
    print("文件不存在")
    os.mkdir("image/" + name)
```

此处 `os` 模块用法同上立绘部分。

(3) 获取所有立绘的图片地址并存入 `image` 文件夹下该角色名称的文件夹中:

```
pictrues = pattern2.findall(con_part)
```

由于官方贺图存在多个, 所以选择调用 `findall` 函数。

```
pattern2 = re.compile("src=\"http.*?\"")
```

由于此处网页中存在其他小图标的 `http` 干扰项, 所以该正则表达式在 `http` 前增加了 `src=` 这一标志。

```
pictrues = pattern2.findall(con_part)

i = 1

for p in pictrues:
    pic = p.replace("src=", "")
```

此处使用 `replace` 函数去掉前面用于标记使用的字符串 “`src=`”

```

        pic = pic.replace("\"", "")

        print(pic)

        r = requests.get(pic)

        f = open("image/" + name + "/" + "官方贺图" + i.__str__() + "." + pic.split('.').pop(),
"wb")

        f.write(r.content)

        print(r.content)

        f.close()

        i = i + 1
    
```

程序运行结果如图 3-6 所示：

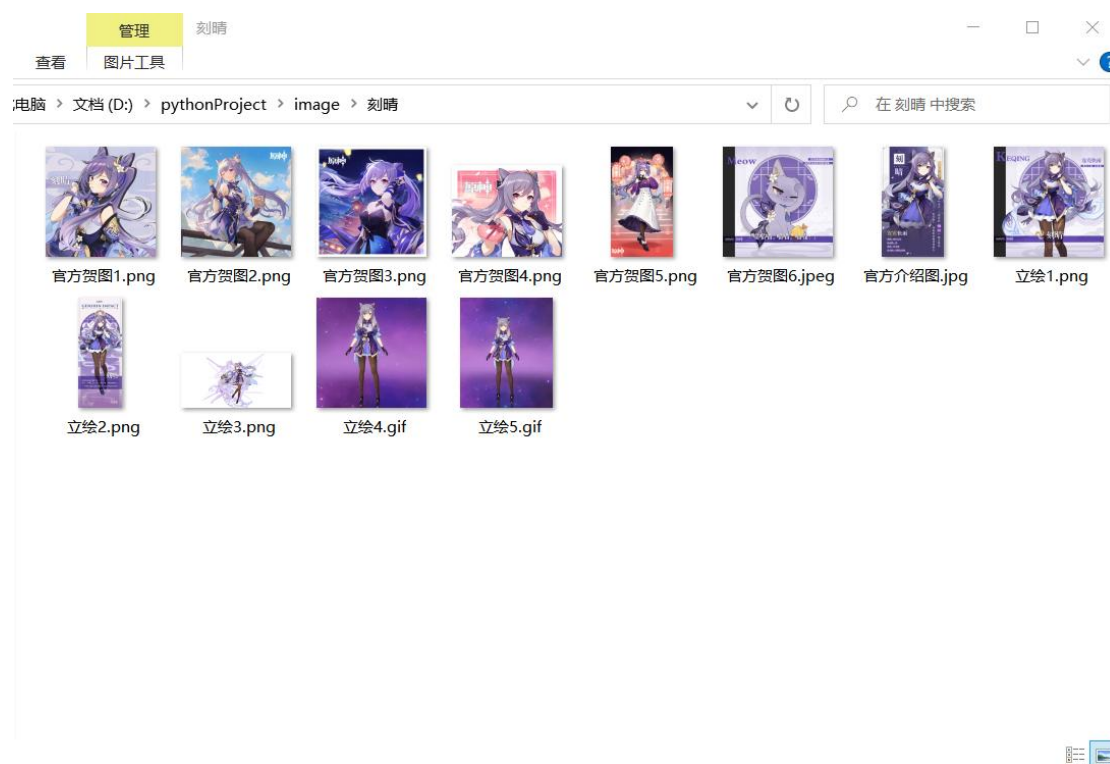


图 3-6 获取单个角色图片

3.2.7 获取单个角色语音

1. 综述

通过前面 3.1.2 所述，当程序想访问每个角色语音网页时需要两部分，一部分是固定的公共网络地址前缀，而另一部分则分别是角色名称+“语音”，通过二者拼接后便可得到完整的角色语音网页地址。实现网页地址的访问后接着就需要对网页格式进行观察。

通过对网页代码的初步观察可以发现，角色语音包含在了一个类似表格的 div 块中，该 div 的类属性为唯一的 class="resp-tabs-container"。而其下还有诸多 div 块，本功能所需 div 块为其内部的第一个类为 class="resp-tab-content" 的 div 块。在此 div 块中存放了角色的所有语音以及语音文本，而下一步则是需要对该 div 块进行进一步解析。

经过观察，该 div 下存在多个类为 class="visible-md visible-sm visible-lg" 的子 div 块，每个子 div 块中都存着一条语音以及对应的语音文本，其 xpath 规则为：

语音标题：./div/div[1]/text()

语音地址：./div/div[2]/div/@data-src

语音文本：./div/div/div[@class="voice_text_chs vt_active"]/text()

通过上述特征便可最终得出功能所需的全部角色语音相关信息。

但是需要注意的是，表格中有些语音标题下没有对应的语音地址链接，此时使用 xpath 对其操作会导致程序崩溃，所以需要采取一定措施解决该错误。

2. 程序片段详解

(1) 访问对应角色语音网址：

```
res1 = requests.get("https://wiki.biligame.com/ys/" + name + "语音")
if str(res1) == '<Response [404]>': # 请求页面不存在
    print('请求 404')
    exit()
```

```
res2 = res1.text.encode('gbk', 'ignore').decode('gbk')
```

(2) 创建该角色在 voicec 路径下的专属文件夹：

```
if not os.path.exists("vioce/" + name):
    print("文件不存在")
```



```
os.mkdir("vioce/" + name)
```

(3) 将获取的网页内容转化为 xpath 对象:

```
html_data = etree.HTML(res2)
```

```
medios=html_data.xpath( '//div[@class="resp-tabs-container"]/div[@class="resp-tab-content"]  
/div[@class="visible-md visible-sm visible-lg"]')
```

该参数表示在全局下先找类名为 resp-tabs-container 的 div 块, 再找其内的第一个类为 resp-tab-content 的 div 块, 最后找出类名为 resp-tab-content 的 div 块下全部类名为 visible-md visible-sm visible-lg 的子 div 块, 并将其数据存于 medios 变量中。

```
for m in medios:
```

```
    title = m.xpath('./div/div[1]/text()')
```

```
    url_m = m.xpath('./div/div[2]/div/@data-src')
```

```
    if url_m and len(url_m[0]):
```

```
        proxies = random.choice(item) # 随机选取一个 IP
```

```
        headers['User-Agent'] = random.choice(user_agent_list)
```

```
        r = get_requests(url_m[0], headers, proxies) #捕捉异常解决连接断开方法
```

由于角色语音数量较多, 可能会导致在爬取过程中被服务器识别出该程序为爬虫, 所以此处采用高匿策略进行角色语音的爬取。

```
        if str(r) == '<Response [404]>': # 请求页面不存在
```

```
            print('请求 404')
```

```
            exit()
```

```
        f = open("vioce/" + name + "/" + title[0] + "." + url_m[0].split('.').pop(), "wb")
```

此处由于写入内容为音频, 所以文件应以二进制格式打开, 属性为 “wb”。

```
        f.write(r.content)
```

```
        f.close()
```

程序运行结果如图 3-7 所示：

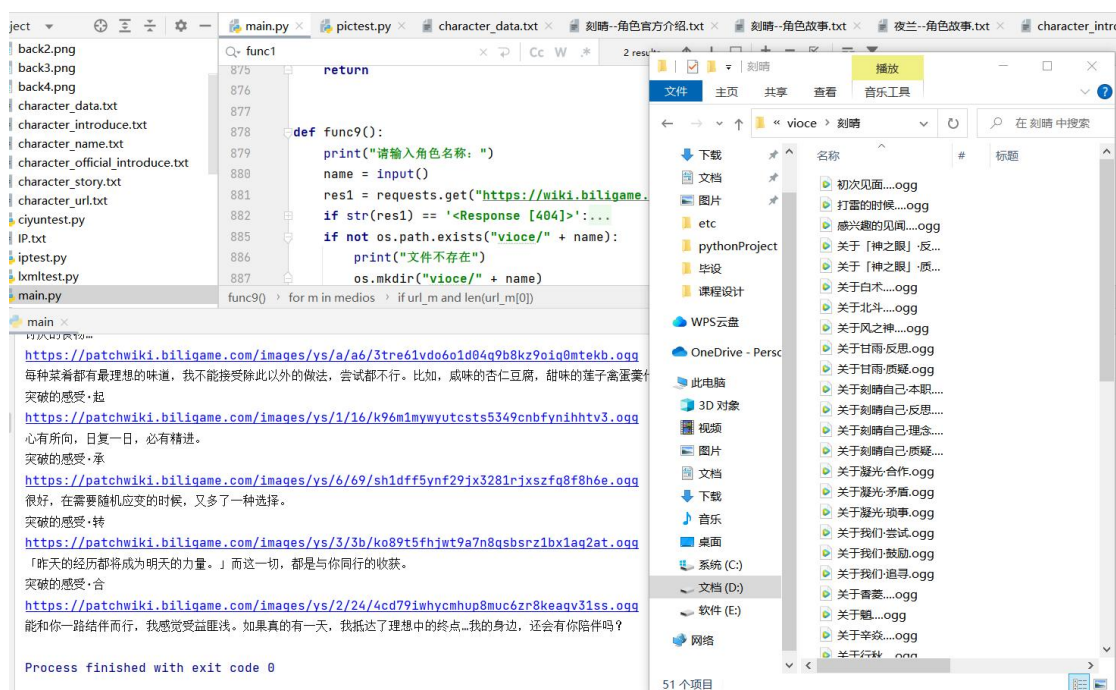


图 3-7 获取单个角色语音

3.3 数据可视化

数据可视化一般来说不是网络爬虫的必要步骤，但它却是一个可以极大提高爬虫价值的步骤，为了使得人们可以更加直观、清晰地理解数据，从而更好地分析、理解和利用数据，直接从网页上获取的直白的数据某种情况下显然无法满足人们的需求，所以如何将数据在人眼观察时体现得更美观，更直白易懂，是十分重要的。

本论文中数据可视化这部分主要包括技术：词云图、csv 表格和柱状图。

3.3.1 角色名称词云

1. 程序源代码

def func1():

file = open("character_name.txt", 'w')

origin_res = requests.get(url2, headers=headers) # 获取网页内容

if str(origin_res) == '<Response [404]>': # 请求页面不存在

```

    print('请求 404')

    exit()

    origin_res_text = origin_res.text.encode('gbk', 'ignore').decode('gbk') # 使网页文本格式
为 gbk,

    origin_res.close()

    html_data = etree.HTML(origin_res_text)

    print(html_data)

    names=html_data.xpath('//div[@class="resp-tabs-container"]/div[@class="resp-tab-content"]
/div[@class="resp-tab-case"]/div/div[@class="L"]/text()')

    for name in names:

        file.write(name+'\n')

        print(name)

    file.close()

#词云部分

text = open('character_name.txt', 'r').read()      cut_text = jieba.cut(text) # 分词处理

word = ''.join(cut_text) # 以空格分割文本

# 读取图片

pic = np.array(Image.open('fire.png'))

image_colors = ImageColorGenerator(pic) # 生成图片颜色中的颜色

wd = wordcloud.WordCloud(

    background_color='white',

    width=4000,

    height=2000,

    margin=2,

    max_words=70, # 设置最多显示的词数

    mask=pic, # 设置词云形状

    font_path="simhei.ttf", # 中文词图必须设置字体格式, 否则会乱码, 这里加载的
是黑体

    random_state=20

)

```

```
wd.generate(word) # 生成词云

# 图片颜色渲染词云图的颜色, 用 color_func 指定

plt.imshow(wd.recolor(color_func=image_colors), interpolation='bilinear')

plt.axis('off') # 关闭显示 x 轴、y 轴下标

plt.show()

print("已将角色名称存入文件 character_name.txt ")

return
```

2. 功能综述

上述代码实现的功能为, 将角色名称信息以 txt 文本格式存入本地文件后, 通过使用 jieba 库的分词统计功能以及一张提前准备好的图片 (wind.png), 再结合 wordcloud 库强大的的绘图功能, 将存在本地的角色名称 txt 文档中的文本读取出来后生成词云图, 并且能够展示出来以及保存。

3. 程序片段详解

(1) 保存角色名称到本地 txt 文件中:

```
file = open("character_name.txt", 'w')

for name in names:

    file.write(name+'\n')

file.close()
```

调用 for 循环将角色名称列表中的角色名称以重写的方式“w”依次写入“character_name.txt”。

(2) 对所有的角色名称使用 jieba 模块下的 cut 函数进行分割:

```
text = open('character_name.txt', 'r').read()

cut_text = jieba.cut(text) # 分词处理

word = ''.join(cut_text) # 以空格分割文本
```

(3) 背景图片读取

```
pic = np.array(Image.open('fire.png'))

image_colors = ImageColorGenerator(pic) # 生成图片颜色中的颜色
```

此处使用到了 NumPy 库中的 array 方法, np.array 是 NumPy 库中的一个函数, 用于创建多维数组。它可以接受一个列表或元组作为参数, 并将其转换为 NumPy

数组对象。

还有 PIL 库中的 open 函数。Image.open() 是 Python Imaging Library(PIL) 中的一个函数,用于打开并加载一张图片。它接受一个图片文件的路径作为参数,返回一个 Image 对象。

以及 ImageColorGenerator 库, ImageColorGenerator 用于从图像中提取颜色信息,并生成颜色调色板。它可以与 PIL (Python Imaging Library) 一起使用,以便从图像中提取颜色信息并生成颜色调色板。

(4) 设置词云图生成参数

```
wd = wordcloud.WordCloud(
    background_color='white',
    width=4000,
    height=2000,
    margin=2,
    max_words=70, # 设置最多显示的词数
    mask=pic, # 设置词云形状
    font_path="simhei.ttf", # 中文词图必须设置字体格式,否则会乱码,这里加载的是黑体
    random_state=20
)
```

上述方法为设置词云的参数。

```
wd.generate(word)
```

该句调用 wordcloud 库的 generate()方法来根据文本生成词云,传入的参数即为全部角色名称转换后的字符串类型。

(4) 制图保存与展示

```
plt.imshow(wd.recolor(color_func=image_colors),interpolation='bilinear')
```

其中第一个参数内函数含义为图片颜色渲染词云图的颜色,用 color_func 指定。

```
plt.axis('off')
```

```
plt.show()
```

第一句功能为图像的处理和显示的格式。

第二句功能为关闭显示 x 轴、y 轴下标，否则可能会影响美观。

第三句功能为图片呈现。

其中 `imshow()` 和 `show()` 有一定的区别。`imshow()` 函数主要是用于进行图像的处理并指定显示的格式，其并不能直接呈现在屏幕上。`show()` 函数则是用于将处理完的图片呈现在屏幕上。

4. 程序运行截图

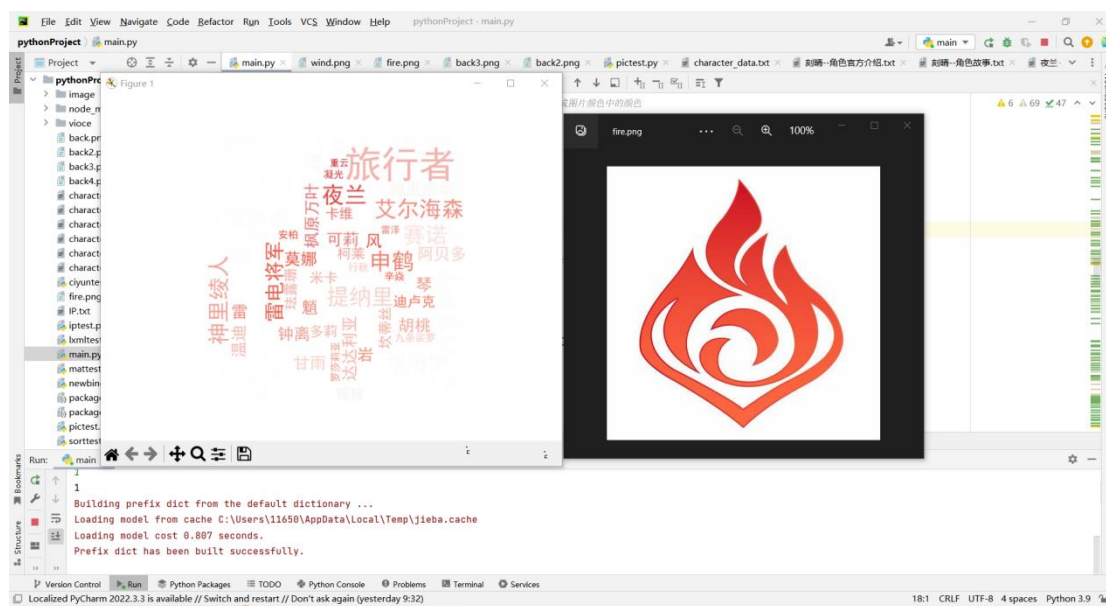


图 3-8 角色名称词云图

3.3.2 角色属性 csv 表格

1. 功能综述

上述代码实现的功能为，将 3.2.5 中获取的角色属性数据通过使用 `csv` 模块使得角色的各个数据属性能够以 `csv` 表格的形式呈现出来。使得角色属性数据更加清晰，易读。

2. 程序片段详解

(1) 数据初始化：

```
file = open(name + "--角色属性.csv", 'w', encoding='utf-8')
```

```
writer = csv.writer(file)
```

此处使用到了 csv 模块。csv 是一种文件格式，一般也被叫做逗号分隔值文件，可以使用 Excel 软件或者文本文档打开。其中数据字段用半角逗号间隔（也可以使用其它字符），使用 Excel 打开时，逗号会被转换为分隔符。csv 文件是以纯文本形式存储了表格数据，并且在兼容各个操作系统。

csv.writer 函数是 Python 中用于将数据写入 CSV 文件的函数。它可以将数据按照指定的格式写入 CSV 文件中，使得数据可以被其他程序或者人类读取和处理。

```
res3 = get_character_data(res2)
```

```
res4 = res3.split(';')
```

get_character_data 函数详见 3.2.4。由于在 get_character_data 函数中处理角色属性数据时已经处理好，分号为表格中行与行之间的间隔符，逗号为每行中不同数据的间隔符，所以此处可直接通过 split 函数获取各行数据。

（2）进行数据分析提取以及写入文件：

i = 0 #i 表示为当前是第几行，从 0 开始计数

```
cheader = [] #表格里的表头
```

```
per = " #特殊属性中的百分比
```

```
while res4:
```

```
    cdata = []
```

```
    if i == 0:
```

```
        head = res4[0].split(',')
```

```
        cheader.append(head[0])
```

```
        head.pop(0)
```

```
        for h in head:
```

```
            cheader.append(h + '突破前')
```

```
            cheader.append(h + '突破后')
```

```
        writer.writerow(cheader)
```

当行号为 1 时，行内存的数据为角色的等级，此时将每个等级突破前，突破后的字符串写入 csv 文件中当作表头。Writer 对象的 writerow() 方法接受一

个列表参数。列表中的每个词，放在输出的 CSV 文件中的一个单元格中。
writerow()函数的返回值，是写入文件中这一行的字符数（包括换行字符）。

函数 pop 可以将该列表中指定行给弹出列表，返回值为被弹出的行。

```
elif i == 1:
    res4.pop(0)
    i = i + 1
    continue
```

当行号为 2 时，行内存的数据为“突破前”、“突破后”，没有意义，可直接跳过。

```
elif i == 2:
    d = res4[0].split(',')
    per = d.pop()
```

此处的 d 为第三行的数据列表，split(',') 得来。pop 函数默认弹出列表中最后一位，该语句将特殊属性中的百分比赋给了 per 这一变量。

```
cdata.append(d[0])
d.pop(0)
for dd in d:
    cdata.append(dd)
    cdata.append(dd)
cdata.append(per)
cdata.append(per)
```

由于第三行是等级 1 的数据，该行所有属性数据与其他行数据不同，等级 1 时突破前和突破后相同且通过爬虫爬取到的数据也只有一份，因此该行数据都得重复加入数据列表两次。

```
writer.writerow(cdata)
elif i == 3:
    d = res4[0].split(',')
    cdata.append(d[0])
```



```
d.pop(0)

for dd in d:

    cdata.append(dd)

cdata.append(per)

cdata.append(per)

writer.writerow(cdata)
```

由于第四行是等级 20 的数据，该行的特殊属性数据与其他行数据不同，等级 20 时突破前和突破后相同且通过爬虫爬取到的特殊属性数据也有一份，因此该行的特殊属性数据需要重复加入数据列表两次。

```
else:

    d = res4[0].split(',')

    for dd in d:

        cdata.append(dd)

    writer.writerow(cdata)
```

其余行正常添加后写入 csv 文件即可。

```
res4.pop(0) #弹出当前行

i = i + 1 #行数+1

file.close()
```

4. 程序运行截图

	A	B	C	D	E	F	G	H	I	J
1	等级	生命上限突破前	生命上限突破后	攻击力突破前	攻击力突破后	防御力突破前	防御力突破后	暴击率突破前	暴击率突破后	暴击率突破后
2										
3	1	1125	1125	19	19	43	43	0%	0%	
4										
5	20	2918	3883	49	66	111	147	0%	0%	
6										
7	40	5810	6495	98	110	220	246	0%	4.80%	
8										
9	50	7472	8386	126	42	283	318	4.80%	9.60%	
10										
11	60	9374	10059	158	170	355	381	9.60%	9.60%	
12										
13	70	11056	11741	187	198	419	445	9.60%	14.40%	
14										
15	80	12749	13434	215	227	483	509	14.40%	19.20%	
16										
17	90	14450	-	244	-	548	-	19.20%	-	
18										
19										

图 3-8 角色属性 csv 表格

3.3.3 角色属性柱状图（单个属性 top 榜）

1. 功能综述

本功能通过进一步解析 3.2.5 获取的角色属性数据排列出了所有角色某一基础属性的各个等级以及突破前后的 top1，并将其以柱状图的形式展示出来。

2. 程序片段详解

由于该功能涉及两部分，代码量过多不便展示，就只挑出部分核心代码进行解读。

（1）数据初始化：

```
levels = ["1 级突破前", "1 级突破后", "20 级突破前", "20 级突破后", "40 级突破前",
"40 级突破后", "50 级突破前", "50 级突破后", "60 级突破前", "60 级突破后", "70 级突破前",
"70 级突破后", "80 级突破前", "80 级突破后", "90 级突破前", "90 级突破后"]

tops = dict(zip(levels, characters))

names = get_character_url(origin_res_text) # 所有角色的网页后缀
```

此处 `get_character_url` 为获取全体角色名称，以便后续遍历爬取全部角色属性数据时使用。

本功能由于数据复杂，所以创建了一以上述 `levels` 列表为 key，自定义类型“TopCharacter”的列表 `characters` 为 value 的一个字典用于存储。自定义类“TopCharacter”中包含了角色名称以及某等级下该角色某个属性的数值。

（2）人机交互选择属性：

```
while 1:

    print("请选择一个属性:")

    print("1、生命上限")

    print("2、攻击力")

    print("3、防御力")

    choose = input()

    if choose == '1' or choose == '2' or choose == '3':

        break
```

此处为手动输入选择，体现了本程序的人机交互性。if 语句使得程序仅有输

入内容为给出的限定字符时才会继续执行程序，否则将重复提示进行选择，不会以错误字符选择执行后续功能，体现出了程序的健壮性。

(3) 开始遍历访问每个角色的网址：

```
for name in names:

    proxies = random.choice(item) # 随机选取一个 IP

    headers['User-Agent'] = random.choice(user_agent_list)

    res1=get_requests(origin_url+name,headers=headers,proxies=proxie)
```

上述代码的功能为开始遍历所有角色名称，并通过本程序封装的 get_requests 函数爬取当前角色的页面信息，get_requests 函数内容详见 3.1.1。后续代码也是在该 for 循环下进行各个角色属性信息的存储和排序判断。

(4) 初步解析判断：

```
chinese_name = urllib.parse.unquote(name)

if tops.keys().__contains__(chinese_name): # 已经有该角色了

    continue

if res3 is None: # 特殊情况：派蒙没有属性

    continue
```

这三行代码功能为，将 url 编码格式下的角色名称通过 urllib 模块下 parse.unquote 函数将其解码为中文文字。之后的第一个 if 语句用来判断榜单列表中是否已经有该成员，这是因为在爬取所有角色名称时可能由于网页内容不严谨的原因，某个角色名称会重复出现，如若该角色已经存在于榜单中则不必继续进行后续属性数据处理。第二个 if 语句则是考虑到派蒙没有属性这一问题。

(5) 针对前面所选属性进行解析，此处以选项 1 生命上限为例，后续代码也是在该 if 语句下：

```
if choose == '1': # 生命上限（第二第三列）

    i = 0

    for r in res4:

        datas = r.split(',')
```

上面的 if 函数表示用户选择的属性为生命上限，对应角色属性表单中的第二第三列。i 变量则表示当前为哪个等级以及突破前还是突破后，i 对应突破前，i+1 对应突破后。

(6) 根据不同等级不同页面结构情况分别进行处理：

```
if i == 0: # 1 级突破前后属性一样
    data_beforeandafter = float(datas[1]) # 第二列
    if tops[levels[i]].getData() < data_beforeandafter:
        tops[levels[i]].setData(data_beforeandafter) # 突破前
        tops[levels[i]].setName(chinese_name)
        tops[levels[i + 1]].setData(data_beforeandafter) # 突破后
        tops[levels[i + 1]].setName(chinese_name)
    # print("等级 1")
    i = i + 2 # 下一级
```

当角色为等级 1 时存在一特殊情况，其突破前后属性一致且在表单中仅占一列，所以使用了 data_beforeandafter 这一变量存储值。当第二个 if 判断成立时表示当前等级下该属性数值更高的角色出现，此时字典列表里对应等级的角色也就发生更换。

```
elif i == 14: # 90 级突破前后属性一样
    data_beforeandafter = float(datas[1]) # 第二列
    if tops[levels[i]].getData() < data_beforeandafter:
        tops[levels[i]].setData(data_beforeandafter) # 突破前
        tops[levels[i]].setName(chinese_name)
        tops[levels[i + 1]].setData(data_beforeandafter) # 突破后
        tops[levels[i + 1]].setName(chinese_name)
```

当角色为等级 90 时存在与等级 1 相似的问题，以相似手段处理即可。

```
else:
    data_before = float(datas[1]) # 第二列
    data_after = float(datas[2]) # 第三列
```

```

if tops[levels[i]].getData() < data_before: # 突破前

    tops[levels[i]].setData(data_before)

    tops[levels[i]].setName(chinese_name)

if tops[levels[i + 1]].getData() < data_after: # 突破后

    tops[levels[i + 1]].setData(data_after)

    tops[levels[i + 1]].setName(chinese_name)

i = i + 2 # 下一级

```

等角色为其他等级时便依照表单位置二三列对应判断处理即可。i=i+2 表示进入下一等级。

后续代码为选择了其他属性下对表格中不同列以及特殊情况下数据的对比和处理，其分析过程与生命上限（choose 为 1 时）几乎没有区别，此处就不再赘述。

（7）将数据以柱状图形式显示出来：

```

plt.style.use('ggplot') # 设置绘图主题为 ggplot

plt.figure(figsize=(14, 7))

plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置图像字体为黑体，同时
可以显示中文

top_datas = []

for l in levels:

    top_datas.append(tops[l].getData())

plt.bar(levels, top_datas)

for l in levels: # 使用 text 函数给柱状图上增加标签

    plt.text(l, tops[l].getData(), tops[l].getName() + "\n" + tops[l].getData().__str__(),
ha='center', va='bottom')

if choose == '1':

    plt.title("各等级突破前后生命上限 top1")

elif choose == '2':

    plt.title("各等级突破前后攻击力 top1")

elif choose == '3':

```

```
plt.title("各等级突破前后防御力 top1")
```

```
plt.show()
```

上述代码使用到了 matplotlib.pyplot 这一模块用于绘制柱状图。plt.style.use 函数功能为设置柱状图的风格; plt.figure 功能为设置画布大小; plt.bar(levels, top_datas) 的功能是将第一个参数作为 x 轴, 第二个参数作为 y 轴生成柱状图, 之后再通过 plt.text 给柱状图的每条数据给上标签, 最后通过 plt.show 函数将其在页面上展示出来。

该功能执行结果截图如图 3-9

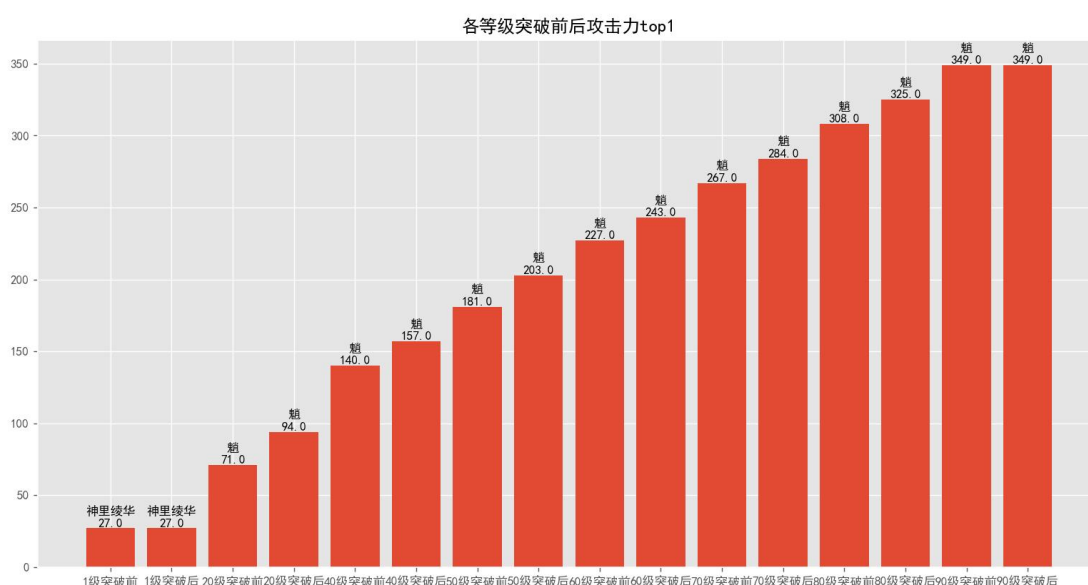


图 3-9 单个属性 top 榜

3.3.4 角色属性柱状图（某一属性某个等级 top 榜）

1. 功能综述

本功能通过进一步解析 3.2.5 获取的角色属性数据排列出了所有角色某一属性某个等级 top5 榜, 并将其以柱状图的形式展示出来。

2. 程序片段详解

由于该功能涉及两部分, 代码量过多不便展示, 就只挑出部分核心代码进行

解读。

(1) 数据初始化:

```
levels = ["", "1 级", "20 级", "40 级", "50 级", "60 级", "70 级", "80 级", "90 级"]
ab = ["", "突破前", "突破后"]
attr = ["", "生命上限", "攻击力", "防御力"]
top_names = ["1", "2", "3", "4", "5"]
top_datas = [0.0, 0.0, 0.0, 0.0, 0.0]
tops = dict(zip(top_names, top_datas))
min_data = tops["1"]
min_name = "1" # 初始为 1
```

以上为功能所需数据结构。levels 列表有 8 个对应需要突破的等级；ab 列表表示突破前还是突破后；attr 列表存放三个基础属性；tops 则是一个字典，用于存放榜单前五的角色及其数据，其 key 为 top_names，value 为 top_datas；min_data 和 min_name 则为临时变量，用于比较当前角色和榜单中角色数据。

(2) 人机交互选择属性、等级以及突破前后:

```
while 1:
    print("请选择一个属性:")
    print("1、生命上限")
    print("2、攻击力")
    print("3、防御力")
    attr_choose = input()
    if attr_choose == '1' or attr_choose == '2' or attr_choose == '3':
        break
```

该部分可参考 3.3.3 中人机交互部分，而等级以及突破前后的选择与此也相似，因此就不再赘述。

(3) 开始遍历爬取所有角色属性数据:

```
for name in names: # 开始爬取某一角色数据并提取出上述选择的数据
    proxies = random.choice(item) # 随机选取一个 IP
```

```
headers['User-Agent'] = random.choice(user_agent_list)
```

`res1 = get_requests(url4 + name, headers, proxies)` # 捕捉异常解决连接断开方法

以上为遍历所有角色名称的过程，并通过本程序封装的 `get_requests` 函数爬取当前角色的页面信息，`get_requests` 函数内容详见 3.1.1。

后续代码也是在该 `for` 循环下进行各个角色属性信息的存储和排序判断。

(4) 初步解析判断：

```
chinese_name = urllib.parse.unquote(name)
```

```
if tops.keys().__contains__(chinese_name): # 已经有该角色了
    continue
```

```
if res3 is None: # 特殊情况：派蒙没有属性
    continue
```

该部分解释详见 3.3.3 中初步解析判断。

(5) 找出 `tops` 字典中最小值，以便后续替换：

```
for m in tops.keys(): # 找出最小值，后续替换掉
```

```
if min_data > tops[m]:
    min_name = m
    min_data = tops[m]
```

其中 `min_name` 和 `min_data` 为用于比较的当前角色信息中间变量。

(6) 根据人机交互时所选项对爬取到的角色属性数据进一步处理，此部分为数据处理的核心部分，，此处以生命上限为例：

```
if attr_choose == '1': # 生命上限
```

```
if ab_choose == '1': # 突破前 应读取第 2 列
```

```
now_data = float(row[1])
```

```
if now_data > min_data:
```

```
    tops[chinese_name] = now_data # 插入新的数据
    tops.pop(min_name)
```



```
min_data = 999999 # 重置数值，以便重新插入新值和删除上一个最小值后找出新的最小值
```

以上这段代码为突破前属性处理代码，其中对于 attr_choose 的判断决定了是哪个属性， ab_choose 则决定了是突破前还是突破后。之后使用 float 函数将表格中的字符串数据转化为浮点类型数据以便后续对比。对比完后将较大值存在 tops 中。

```
if ab_choose == '2': # 突破后 应读取第 3 列
    if level_choose == '1' : # 1 级和 90 级还是第 2 列
        now_data = float(row[1])
    elif level_choose == '8':
        now_data = float(row[1])
    else:
        now_data = float(row[2])
    if now_data > min_data:
        tops[chinese_name] = now_data # 插入新的数据
        tops.pop(min_name)
    min_data = 999999 # 重置数值，以便重新插入新值和删除上一个最小值后找出新的最小值
```

此段代码为突破后属性处理代码，处理过程与突破前基本相同，区别等级 1 和等级 90 突破前后都为同一列数据，其他等级突破前为第 i 列的话，突破后为第 i+1 列。

(7) 将数据以柱状图形式显示出来：

```
plt.style.use('ggplot') # 设置绘图主题为 ggplot
plt.figure(figsize=(14, 7))
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置图像字体为黑体，同时可以显示中文

plt.bar(sorted_tops_key, sorted_tops_value, bottom=min_data-10)
for k in sorted_tops_key: # 使用 text 函数给柱状图上增加标签
    plt.text(k, tops[k], tops[k], ha='center', va='bottom')
```

```
plt.title(title)
```

```
plt.show()
```

上述代码中 `plt` 的函数使用同 3.3.3 中柱状图显示，区别在于其 `plt.bar` 和 `plt.text` 中给定数据不同。

该功能执行结果截图如图 3-10，此处选择等级 80 突破后，属性防御力作为例子：

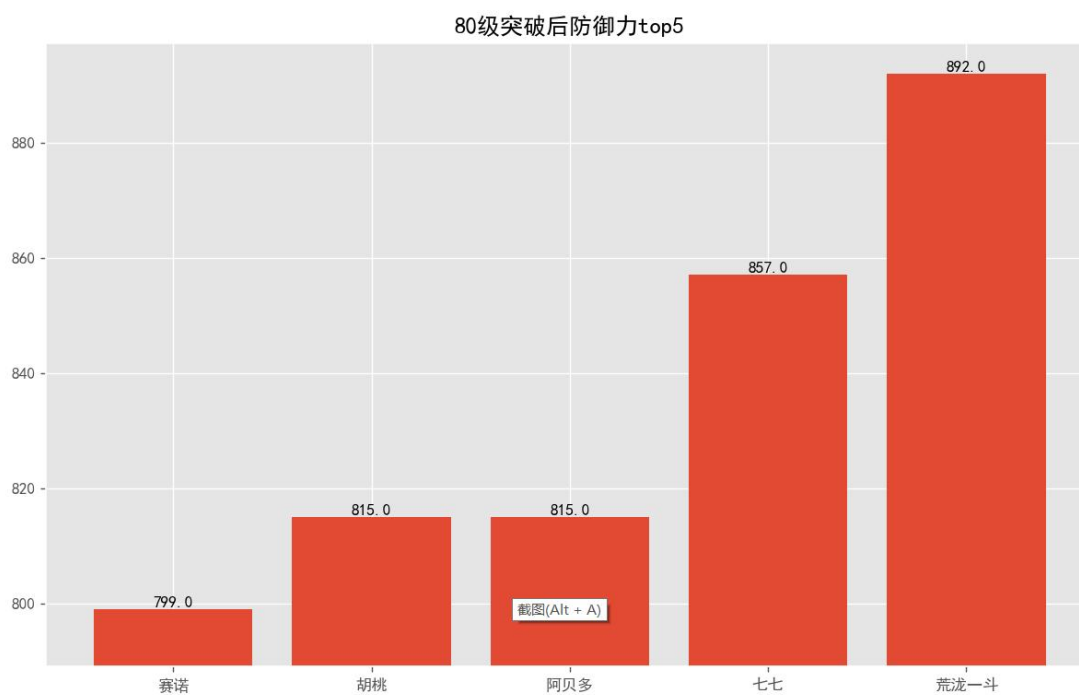


图 3-10 某一属性某个等级 top 榜

第 4 章 结论

4.1 系统存在问题

(1) 本系统数据可视化部分还是略显简陋，虽然可以基本满足目标功能的实现，但整体而言美观度以及操作简洁度还有待提高。

(2) 数据存储于本地的方式都采取的是 `txt`、`csv` 等文档的形式用于保存而不是专业的数据库，虽然相比数据库方便简洁，但对于数据存储而言并不够正式，安全性也有待商榷。

(3) 爬取数据的过程中并没有遭遇到网站对系统进行“封锁 `ip`”，“登录账号”，“人机验证”之类的事情，所以本系统对于这些人机验证方面的处理和应用并无太多涉及。

(4) 数据解析时使用的是较为原始的 `re` 正则表达式模块为主，`lxml` 模块为辅，而不是 `BeautifulSoup`、`Scrapy` 和 `Crawley` 等更加新颖的库。

4.2 功能完成情况

本次基于 `python` 网络爬虫的设计与实现选择了原神 Wiki 作为爬取对象，对其中的角色名称、角色简介、角色官方介绍、角色故事、角色攻击力属性、防御力属性、生命上限属性、角色图片、角色语音等信息进行了爬取，由于获取其他角色相关数据与获取以上数据的过程比较相似，同时也是为了限制篇幅，所以本次仅对以上数据进行了爬取。该网络爬虫系统基本上达到了一个 `python` 网络爬虫系统应有的功能，例如获取目标网页，解析目标页面，获取所需数据，保存处理后数据，读取数据以及数据的可视化。解析网页方面使用到了 `re` 正则表达式模块和 `lxml` 模块。数据可视化方面则是通过词云、`csv` 表格以及柱状图这三种方式完成数据的呈现。

4.3 个人成长心得

在完成这个 python 爬虫的毕业设计后，我对于网络爬虫方面的知识有了更进一步的了解。在这个过程中我学会了如何准确的获取我需要的网页，如何分析我获取到的页面，甚至对于网站的页面结构有了更多的认识 and 了解。同时，由于网络爬虫需要对网页上的各种繁杂的数据进行筛选分析，因此毕业设计也极大锻炼了我对于信息进行提炼分类总结的能力。

参考文献

- [1] 吴永聪.浅谈 Python 爬虫技术的网页数据抓取与分析[J]. 计算机时代,2019(08)
- [2] 池毓森.基于 Python 的网页爬虫技术研究[J]. 信息与电脑(理论版),2021(21)
- [3] 郭二强;李博.大数据环境下基于 python 的网络爬虫技术[J]. 计算机产品与流通,2017
- [4] 孙俊,李秋月,赵晨悦.基于 Python 的图书信息搜集与展示研究[J]. 现代信息科技,2022(05)
- [5] 温娅娜,袁梓梁,何咏宸,黄猛.基于 Python 爬虫技术的网页解析与数据获取研究[J], 现代信息科技,2020(01)
- [6] 谢克武.大数据环境下基于 python 的网络爬虫技术[J]. 电子制作,2017(09)
- [7] 杨孟姣,杜棋东.基于 Python 爬虫网站数据分析系统设计与实现[J].计算机时代,2022(11)
- [8] 韦依洋,吴一凡,李永远.Python 技术在数据可视化中的应用研究[J]. 福建电脑,2022(01)
- [9] 徐志,金伟.Python 爬虫技术的网页数据抓取与分析[J]. 数字技术与应用,2020(10)
- [10] 成文莹,李秀敏.基于 Python 的电影数据爬取与数据可视化分析研究[J]. 电脑知识与技术,2019(31)
- [11] 罗刚,王振东.自己动手写爬虫[M]. 北京:清华大学出版社,2010:32-130.
- [12] 彭嫚.面向主题的 web 信息采集系统的设计与实现[J]. 信息与电脑(理论版). 2014.
- [13] Mali, S.,Meshram, B.B.. Implementation of multiuser personal web crawler[P]. Software Engineering (CONSEG), 2012 CSI Sixth International Conference on,2012.
- [14] Winter. 中文搜索引擎技术解密[M]. 北京:北京邮电出版社,2004:26-110.
- [15] Saed ALQARALEH,Omar RAMADAN,Muhammed SALAMAH. Efficient watcher based web crawler design[J]. Aslib Journal of Information Management,2015,67(6).
- [16] 崔庆才. Python3 网络爬虫开发实践[M]. 北京:中国工信出版社&人民邮电出版社,2018:1-280.
- [17] Dusan Stevanovic,Aijun An,Natalija Vlajic. Feature evaluation for web crawler detection with data mining techniques[J]. Expert Systems With Applications,2012,39(10).
- [18] 司维,曾军威,谭颖华译. Python 基础教程(第二版 修订版) [M]. 北京:人民邮电出版社,2014:26-137.
- [19] 张浩. 网络主题爬虫技术初探[J]. 电子制作. 2013.
- [20] 陈乐.基于 Python 的网络爬虫技术[J].电子世界,2018(16):163-165.

致 谢

时光易逝，我的大学生涯也即将结束。于此，我想感谢父母，为我支撑起坚实的护盾，在我低迷时鼓励我前行；感谢母校，为我们提供了良好的学习环境；感谢辅导员王星阳老师，在大学四年里指引我们前行；感谢舍友，在平常的日子里一同欢笑，一同进步；最后感谢论文导师裴士辉老师，给予了我完成毕业论文的极大支持和帮助。这四年里有欢笑，有泪水，有艰辛，有感动。在这度过的每时每刻我都将珍藏在我的记忆中，伴我成长。